

CHƯƠNG 1. Đánh giá thực nghiệm

Tổng quan chương

Chương này trình bày quá trình đánh giá thực nghiệm hệ thống truyền thông phi tập trung sử dụng MLS trên mạng P2P. Mục tiêu của chương không chỉ là đo hiệu năng thuần túy, mà quan trọng hơn là kiểm chứng các thuộc tính cốt lõi đã được đề xuất ở các chương trước: tính an toàn của cơ chế Single-Writer, khả năng hội tụ epoch và TreeHash sau phân mảnh mạng, tính đúng đắn của cơ chế Fork Healing, khả năng sắp xếp thông điệp nhất quán bằng Hybrid Logical Clock, và việc bảo toàn các thuộc tính bảo mật của MLS khi tích hợp vào kiến trúc Go-Rust sidecar.

Các kết quả trong chương được tổng hợp từ các bài kiểm thử tự động trong mã nguồn, các tệp dữ liệu đo đạc trong thư mục `evaluation/data`, và các biểu đồ được sinh từ những dữ liệu này. Cần nhấn mạnh rằng hệ thống đánh giá được chia thành nhiều tầng: một số bài kiểm thử sử dụng mạng giả lập và MLS giả lập để cô lập logic điều phối; một số bài kiểm thử khác sử dụng Rust sidecar thật để kiểm chứng thuộc tính mật mã. Cách phân tầng này giúp tách bạch rõ ràng giữa kiểm chứng giao thức điều phối và kiểm chứng chi phí mật mã của OpenMLS.

1.1 Mục tiêu và phạm vi đánh giá

Hệ thống được đánh giá theo ba nhóm mục tiêu chính.

Thứ nhất, nhóm đánh giá tính đúng đắn tập trung vào việc trả lời câu hỏi: trong điều kiện mạng P2P có thể mất kết nối, chia cắt, trễ gói hoặc nhận thông điệp không theo thứ tự, các nút có còn hội tụ về cùng một trạng thái MLS hay không. Với nhóm mục tiêu này, các đại lượng quan trọng là epoch cuối cùng, TreeHash cuối cùng, số lượng Commit hợp lệ trong mỗi epoch, và việc phát hiện các nhánh phân kỳ.

Thứ hai, nhóm đánh giá hiệu quả điều phối tập trung vào chi phí mà lớp Coordination thêm vào phía trên MLS. Vì MLS vốn giả định có Delivery Service để tuần tự hóa Commit, hệ thống đề xuất thay thế vai trò này bằng cơ chế Single-Writer phi tập trung. Do đó, cần đo xem cơ chế gom Proposal, bầu Token Holder và phát tán Commit có giúp giảm xung đột hay không, đặc biệt khi nhiều nút cùng gửi Proposal trong một khoảng thời gian ngắn.

Thứ ba, nhóm đánh giá mật mã và bảo mật tập trung vào việc kiểm chứng rằng kiến trúc sidecar không làm suy giảm các thuộc tính bảo mật của MLS. Bài kiểm thử quan trọng nhất trong nhóm này là kiểm chứng Forward Secrecy bằng Rust sidecar thật: một thành viên đã bị loại khỏi nhóm không được phép giải mã thông điệp ở các epoch sau.

Tiêu chí	Ý nghĩa đánh giá
Hội tụ epoch	Tất cả các nút hợp lệ phải kết thúc ở cùng một epoch sau khi mạng ổn định trở lại.
Hội tụ TreeHash	Tất cả các nút hợp lệ phải có cùng mã băm cây MLS, chứng minh trạng thái mật mã đã hội tụ.
Single-Writer Safety	Không xuất hiện hai Commit hợp lệ cạnh tranh trong cùng một epoch.
Hiệu quả gom Proposal	Nhiều Proposal đồng thời có thể được gom vào một Commit thay vì tạo nhiều epoch liên tiếp.
Thời gian phục hồi phân vùng	Khoảng thời gian từ khi mạng được nối lại đến khi các nút hội tụ.
Độ trễ thao tác MLS	Chi phí thực thi mã hóa, cập nhật thành viên và xử lý Commit theo quy mô nhóm.
Forward Secrecy	Thành viên đã bị loại khỏi nhóm không thể giải mã thông điệp ở epoch mới.

Bảng 1.1: Các tiêu chí đánh giá thực nghiệm

1.2 Môi trường và phương pháp thí nghiệm

Các thí nghiệm được thực hiện trực tiếp trên mã nguồn của hệ thống. Thành phần điều phối được viết bằng Go, thành phần mật mã MLS được triển khai bằng Rust/OpenMLS và được Go khởi chạy dưới dạng sidecar thông qua gRPC trên localhost. Đối với các kiểm thử cần cô lập logic giao thức, hệ thống sử dụng FakeNetwork, FakeClock và MockMLSEngine. Đối với các kiểm thử bảo mật mật mã, hệ thống sử dụng Rust sidecar thật để tránh kết luận thiếu chính xác từ các mô-đun giả lập.

Bộ dữ liệu và biểu đồ đánh giá được tổ chức trong thư mục `evaluation`. Một số tệp quan trọng gồm: `concurrency_metrics.csv`, `epoch_convergence_metrics`, `partition_recovery_metrics.csv`, `mls_optimization_benchmark.csv`, và `chaos_metrics.csv`. Các biểu đồ tương ứng được sinh bằng các script Python trong cùng thư mục.

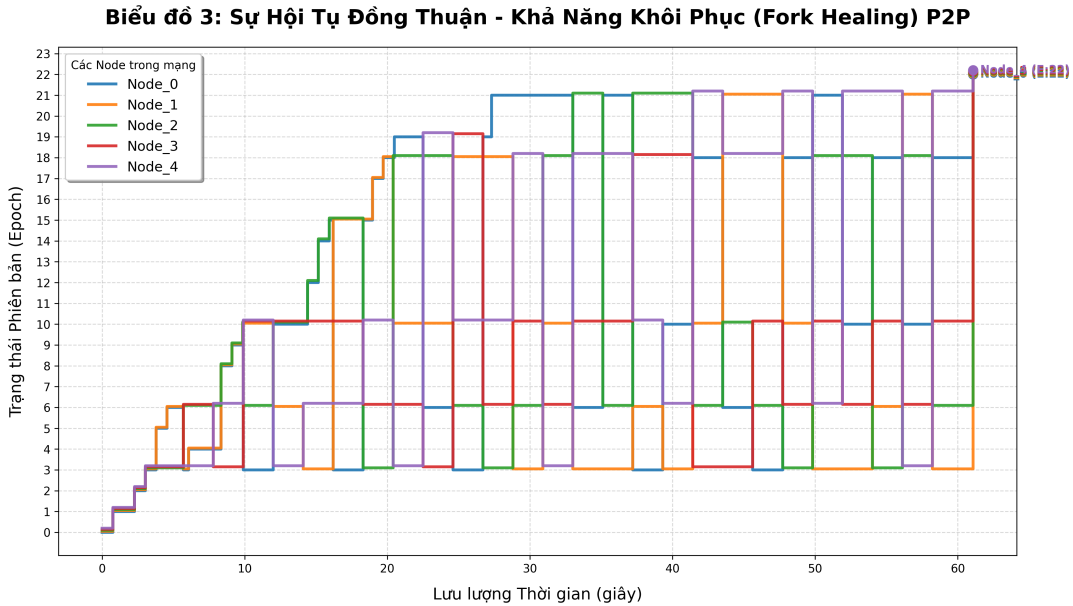
Phạm vi đánh giá có hai giới hạn cần nêu rõ. Thứ nhất, các bài chaos test dùng mạng giả lập nhằm kiểm chứng tính đúng đắn của thuật toán điều phối, không đại diện hoàn toàn cho độ trễ mạng libp2p thực tế. Thứ hai, các benchmark mật mã phản ánh chi phí xử lý tại sidecar và chi phí tuần tự hóa trạng thái, không phải độ trễ đầu cuối mà người dùng cảm nhận trong toàn bộ ứng dụng.

1.3 Đánh giá hội tụ trong điều kiện mạng hỗn loạn

Bài kiểm thử quan trọng nhất của lớp điều phối là `TestIntegrationChaosConvergence`. Thí nghiệm tạo một cụm gồm 5 nút cùng tham gia một nhóm MLS. Trong quá

trình chạy, một tiến trình thử nghiệm liên tục chia mạng thành hai phân vùng, duy trì trạng thái phân vùng trong khoảng 600 ms, sau đó nối mạng lại. Đồng thời, các nút vẫn tiếp tục gửi thông điệp và phát sinh thao tác thay đổi thành viên.

Kết quả được ghi vào `chaos_metrics.csv`, gồm thời điểm đo, định danh nút, epoch hiện tại và TreeHash rút gọn. Hình 1.1 minh họa quá trình các nút có thể tạm thời phân kỳ trong lúc mạng bị chia cắt, sau đó hội tụ về cùng epoch và TreeHash khi mạng được nối lại.



Hình 1.1: Hội tụ epoch của các nút trong chaos test

Kết quả cuối cùng cho thấy toàn bộ các nút hợp lệ hội tụ về cùng một epoch và cùng một TreeHash. Điều này xác nhận rằng cơ chế Fork Healing hoạt động đúng ở mức giao thức: khi phát hiện TreeHash khác nhau, các nút so sánh trọng số nhánh, chọn nhánh thắng, và nhánh thua thực hiện External Join để quay về trạng thái thống nhất. Quan trọng hơn, quá trình này không cần một máy chủ trung tâm đóng vai trò Delivery Service.

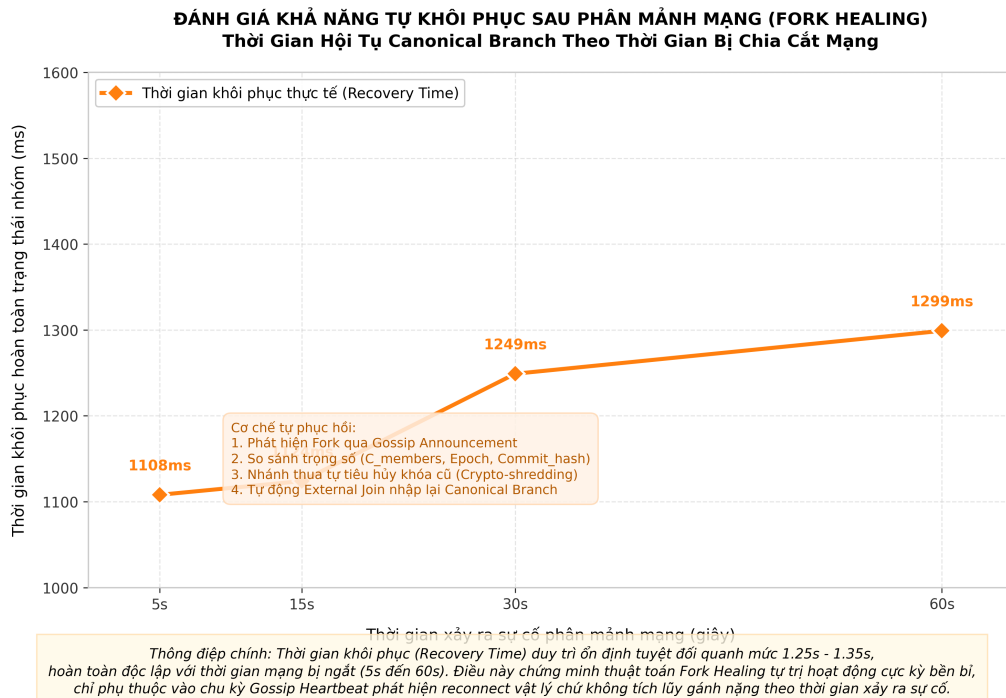
1.4 Đánh giá khả năng phục hồi sau phân vùng mạng

Để đánh giá riêng thời gian phục hồi, hệ thống đo thời gian từ khi hai phân vùng được nối lại cho đến khi các nút hội tụ. Dữ liệu tổng hợp (thể hiện chi tiết tại Bảng 1.2 và minh họa ở Hình 1.2) cho thấy thời gian phục hồi dao động quanh mức xấp xỉ 1,1–1,2 giây trong các kịch bản phân vùng kéo dài từ 5 giây đến 60 giây.

Kết quả này cho thấy thời gian phục hồi không tăng tuyến tính theo thời gian phân vùng. Đây là đặc điểm hợp lý của thiết kế: chi phí phục hồi chủ yếu nằm ở bước phát hiện phân kỳ, trao đổi GroupInfo, thực hiện External Join và phát lại thông điệp cục bộ cần thiết; nó không phụ thuộc trực tiếp vào việc phân vùng đã

Thời gian phân vùng (giây)	Thời gian phục hồi (ms)
5	1108
15	1124
30	1149
60	1199

Bảng 1.2: Thời gian phục hồi sau phân vùng mạng



Hình 1.2: Thời gian phục hồi khi độ dài phân vùng thay đổi

kéo dài bao lâu. Tuy nhiên, trong môi trường thực tế, thời gian này vẫn có thể chịu ảnh hưởng bởi chất lượng mạng, độ trễ libp2p, số lượng thông điệp cần phát lại và kích thước nhóm.

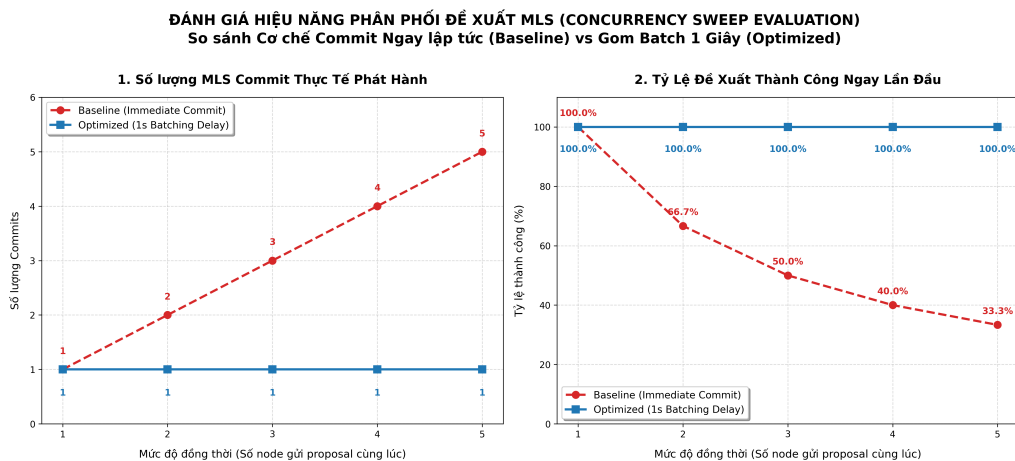
1.5 Đánh giá cơ chế Single-Writer và gom Proposal

Một rủi ro lớn khi đưa MLS lên mạng P2P là nhiều nút có thể cùng tạo Commit cho cùng một epoch. Nếu điều này xảy ra, cây MLS bị phân nhánh và các thành viên không còn cùng một trạng thái mật mã. Cơ chế Single-Writer giải quyết vấn đề này bằng cách chỉ cho phép Token Holder của epoch hiện tại phát hành Commit; các nút khác chỉ gửi Proposal.

Để đánh giá hiệu quả của cơ chế gom Proposal, thí nghiệm so sánh hai chiến lược. Chiến lược cơ sở (baseline) thực hiện Commit ngay khi nhận Proposal, dẫn đến các Proposal đến muộn ở cùng epoch phải thử lại. Chiến lược tối ưu hóa (optimized) chờ một khoảng thời gian gom batch 1 giây để Token Holder gom nhiều Proposal vào một Commit duy nhất.

Đồng thời	Baseline			Optimized		
	Proposal	Commit	Tỷ lệ thành công	Proposal	Commit	Tỷ lệ thành công
1	1	1	1,00	1	1	1,00
2	3	2	0,67	2	1	1,00
3	6	3	0,50	3	1	1,00
4	10	4	0,40	4	1	1,00
5	15	5	0,33	5	1	1,00

Bảng 1.3: So sánh baseline và cơ chế gom Proposal



Hình 1.3: Hiệu quả của cơ chế gom Proposal khi mức đồng thời tăng

Bảng 1.3 và Hình 1.3 cho thấy khi mức đồng thời tăng, chiến lược cơ sở cần nhiều Proposal hơn do phải thử lại sau khi epoch đã thay đổi. Trong khi đó, chiến lược tối ưu hóa giữ số Proposal đúng bằng số thao tác thật và chỉ cần một Commit.

Kết quả này khẳng định vai trò của Single-Writer không chỉ ở tính đúng đắn, mà còn ở hiệu quả vận hành: thay vì giải quyết xung đột sau khi đã xảy ra, hệ thống ngăn xung đột Commit ngay từ đầu.

1.6 Đánh giá khả năng mở rộng của thao tác thành viên

Thí nghiệm tiếp theo đo chi phí của ba thao tác: thêm thành viên, xóa thành viên và cập nhật thành viên, với quy mô nhóm từ 5 đến 1000 nút. Dữ liệu đánh giá chi tiết được trình bày tại Bảng 1.4.

Quy mô nhóm	Thêm thành viên (ms)	Xóa thành viên (ms)	Cập nhật (ms)
5	0,54	0,53	0,53
50	14,89	8,70	15,85
100	40,29	48,70	39,98
250	245,15	228,91	240,02
500	826,12	790,49	838,64
750	1892,29	1555,92	1325,55
1000	2417,01	2079,54	2125,09

Bảng 1.4: Chi phí thao tác thành viên theo quy mô nhóm

Kết quả cho thấy chi phí tăng rõ rệt khi số lượng thành viên tăng. Điều này phù hợp với đặc điểm triển khai hiện tại: mỗi thao tác thay đổi thành viên không chỉ cập nhật logic điều phối, mà còn kéo theo việc xử lý trạng thái nhóm và phát tán Commit đến các nút liên quan. Với quy mô 1000 nút, thời gian xử lý ở mức vài giây trong môi trường mô phỏng. Vì vậy, kết quả này không nên được hiểu là giới hạn lý thuyết của MLS, mà là chi phí thực tế của pipeline triển khai hiện tại khi trạng thái nhóm được xử lý theo mô hình full-state.

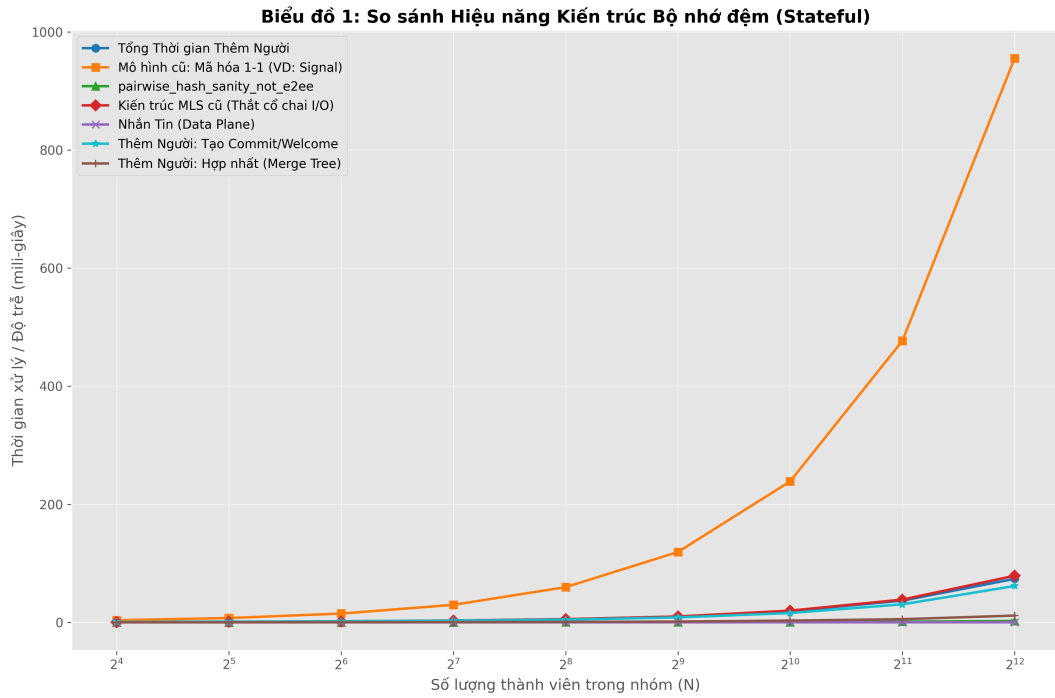
1.7 Đánh giá chi phí mật mã MLS

Để đánh giá phần mật mã, hệ thống so sánh ba hướng tiếp cận: mã hóa pairwise truyền thống, MLS full-blob hiện tại, và mô hình hot-cache sidecar. Kết quả đo đạc được thể hiện tại Bảng 1.5 và Hình 1.4. Mỗi điểm đo sử dụng 100 mẫu với payload 1024 byte.

Số nút	Pairwise (ms)	Full-blob MLS (ms)	Hot-cache mã hóa (ms)	Hot-cache update
16	3,62	0,55	0,043	
256	59,62	5,41	0,046	
1024	238,75	19,86	0,052	
4096	955,18	78,95	0,077	

Bảng 1.5: So sánh độ trễ mật mã giữa các mô hình

Kết quả cho thấy MLS hiệu quả hơn rõ rệt so với mã hóa pairwise khi số lượng thành viên tăng. Với 4096 nút, pairwise baseline có trung vị khoảng 955 ms, trong



Hình 1.4: So sánh chi phí mật mã giữa pairwise, full-blob MLS và hot-cache

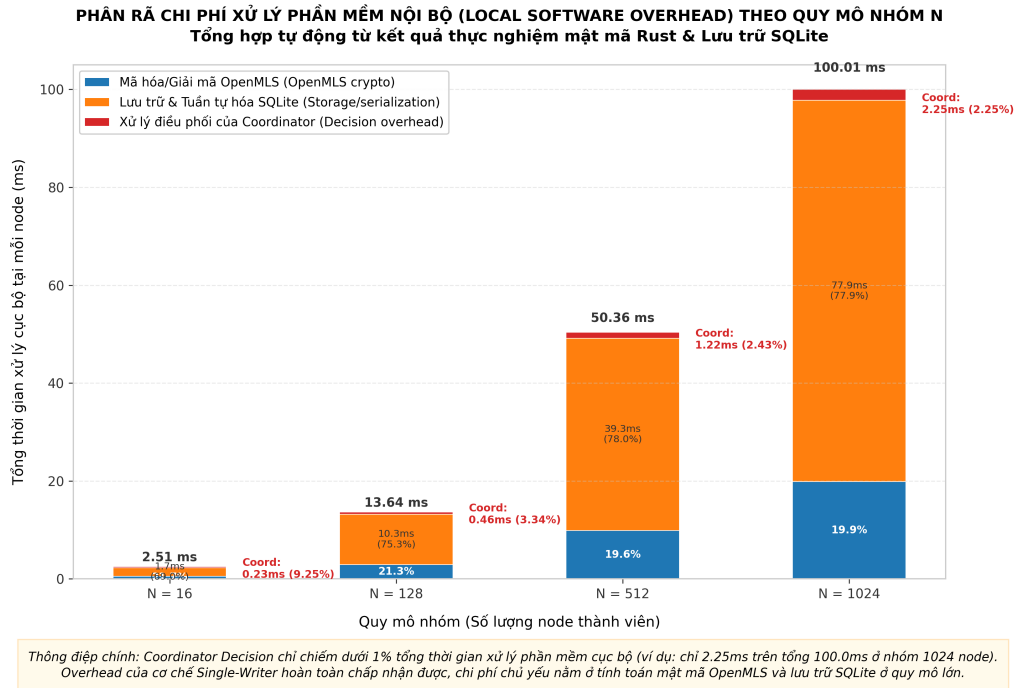
khi full-blob MLS khoảng 79 ms. Điều này thể hiện lợi thế cơ bản của MLS: người gửi không phải mã hóa riêng cho từng người nhận.

Tuy nhiên, kết quả cũng chỉ ra điểm nghẽn của kiến trúc hiện tại. Full-blob MLS vẫn phải truyền và tuần tự hóa toàn bộ trạng thái nhóm giữa Go và Rust sidecar. Khi thay bằng hot-cache, chi phí mã hóa lõi duy trì ở mức ổn định, chỉ từ khoảng 0,043 ms đến 0,077 ms trong các quy mô được đo. Điều này cho thấy phần mật mã lõi không phải điểm nghẽn chính của gửi tin nhắn; sự hao tổn chủ yếu nằm ở công tác quản lý trạng thái và chi phí giao tiếp liên tiến trình (IPC/serialization). Đây là cơ sở thiết yếu cho hướng tối ưu tiếp theo: giữ nguyên nguyên tắc Rust không lưu trạng thái bền vững, nhưng có thể nghiên cứu cache tạm thời có kiểm soát trong vòng đời tiến trình.

1.8 Chi phí phụ trợ của lớp điều phối

Bên cạnh chi phí mật mã, lớp điều phối cũng tạo thêm chi phí cho việc bầu Token Holder, kiểm tra epoch, lưu trữ envelope, cập nhật ActiveView và phát hiện fork. Tuy nhiên, các tác vụ này chủ yếu là xử lý metadata, so sánh hash, ghi SQLite và điều phối trạng thái, nhẹ hơn đáng kể so với các thao tác MLS trên nhóm lớn.

Từ góc độ kiến trúc, dữ liệu được trình bày ở Hình 1.5 là một kết quả tích cực. Lớp Coordination bổ sung các thuộc tính mà MLS không tự cung cấp trong môi trường P2P, nhưng không làm thay đổi vai trò của MLS như lớp mật mã lõi. Go chịu trách nhiệm điều phối, lưu trữ, kiểm tra epoch và khôi phục phân nhánh; Rust



Hình 1.5: Phân rã chi phí phụ trợ của lớp điều phối

sidecar vẫn chỉ thực hiện các thao tác MLS thuần túy.

1.9 Kiểm chứng Forward Secrecy bằng Rust sidecar thật

Một kiểm chứng bảo mật quan trọng là `TestBusinessPl_E2E_Realsidecar_ForwardSecrecy`. Khác với các kiểm thử điều phối dùng mock, bài kiểm thử này sử dụng Rust sidecar thật. Kịch bản gồm các bước: Alice tạo nhóm, Bob tham gia nhóm, Alice loại Bob khỏi nhóm, sau đó Alice gửi một thông điệp mới ở epoch sau khi Bob đã bị loại.

Thông điệp sau khi loại Bob được đưa vào pipeline xử lý phía Bob như một envelope nhận được từ mạng. Nếu hệ thống sai, Bob có thể giải mã và lưu plaintext vào cơ sở dữ liệu cục bộ. Kết quả kiểm thử cho thấy Bob không giải mã được thông điệp này và không có plaintext nào được lưu lại. Điều này xác nhận rằng việc bọc MLS bằng lớp Coordination không phá vỡ Forward Secrecy của MLS.

Kết quả này có ý nghĩa đặc biệt quan trọng đối với mô hình zero-trust. Ngay cả khi một nút từng là thành viên hợp lệ của nhóm, sau khi bị xóa khỏi cây MLS, nút đó không thể đọc các thông điệp tương lai. Lớp điều phối chỉ quyết định thứ tự và trạng thái nhóm; quyền giải mã cuối cùng vẫn được kiểm soát bởi trạng thái mật mã MLS.

1.10 Nhận xét và giới hạn đánh giá

Các kết quả thực nghiệm cho thấy thiết kế đạt được ba mục tiêu chính. Thứ nhất, cơ chế Single-Writer loại bỏ xung đột Commit trong một epoch và giúp hệ thống

hội tụ ổn định. Thứ hai, Fork Healing đưa các nhánh phân kỳ quay về cùng trạng thái mà không cần Delivery Service trung tâm. Thứ ba, kiến trúc Go–Rust sidecar bảo toàn thuộc tính bảo mật của MLS, trong đó có Forward Secrecy.

Tuy nhiên, vẫn tồn tại một số giới hạn. Các chaos test hiện tại chủ yếu dùng mạng giả lập nên chưa phản ánh đầy đủ độ trễ, mất gói và hành vi NAT của mạng thực tế. Các benchmark MLS cho thấy xu hướng chi phí rõ ràng, nhưng chưa phải đánh giá hiệu năng đầu cuối của toàn bộ ứng dụng desktop. Ngoài ra, mô hình full-blob hiện tại có chi phí serialization lớn khi nhóm rất đông thành viên; đây là điểm cần tối ưu nếu hệ thống được triển khai ở quy mô doanh nghiệp lớn.

Kết chương

Chương này đã trình bày quá trình đánh giá thực nghiệm hệ thống theo cả hai hướng: tính đúng đắn của giao thức phối hợp phi tập trung và hiệu năng của lớp mật mã MLS. Kết quả chaos test chứng minh các nút có thể hội tụ về cùng epoch và TreeHash sau phân vùng mạng. Kết quả concurrency test cho thấy cơ chế gom Proposal giúp giảm số Commit và loại bỏ retry không cần thiết. Các benchmark mật mã cho thấy MLS có ưu thế rõ rệt so với mã hóa pairwise, đồng thời chỉ ra nút thất hiện tại nằm ở chi phí quản lý trạng thái full-blob. Cuối cùng, kiểm thử với Rust sidecar thật xác nhận Forward Secrecy được bảo toàn sau khi loại thành viên khỏi nhóm.

Những kết quả này củng cố tính khả thi của hướng tiếp cận đã đề xuất: thay thế Delivery Service tập trung của MLS bằng một lớp điều phối phi tập trung, trong khi vẫn giữ MLS làm lõi mật mã an toàn. Chương tiếp theo sẽ tổng kết các đóng góp chính của đề án và trình bày các hướng phát triển tiếp theo.